Universidad Católica de Temuco
Ingeniería Civil Informática

Teoría de la Computación — INFO1148

Proyecto: Diseño e implementación de un analizador sintáctico

DomoDSL

Un lenguaje de reglas para la automatización de una casa inteligente (IoT)

Gramática libre de contexto · FIRST y FOLLOW · Tabla LL(1)
Parser predictivo · AST · Manejo de errores

Integrantes: Juan Claudio Cárdenas Uribe
Juan Esteban Llanca Hernández
Bayron Rodrigo Flores Riquelme

Docente: Prof. M. Lévano

Fecha: 30 Octubre de 2026

Repositorio: `https://github.com/BayronFlores/domodsl-parser-ll1`

Implementación en Python 3 · sin generadores de parsers · solo biblioteca estándar

1 Introducción y dominio seleccionado

Los lenguajes de programación y los lenguajes de dominio específico (DSL) necesitan mecanismos que verifiquen si una secuencia de símbolos cumple una estructura formal. Este proyecto diseña e implementa, *desde cero*, un analizador sintáctico para un pequeño DSL, aplicando los contenidos del curso: gramáticas libres de contexto (GLC), eliminación de recursión por la izquierda, factorización, conjuntos FIRST y FOLLOW, tabla predictiva LL(1), construcción de árboles sintácticos y detección de errores.

Dominio escogido. Se eligió la **automatización de una casa inteligente (IoT)**, el ejemplo orientativo del enunciado. El usuario describe *reglas* del tipo "cuando ocurra esta condición, ejecuta estas acciones":

```
WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador;
```

Este dominio es adecuado porque (i) las reglas condición–acción son naturales y fáciles de validar, (ii) las condiciones admiten conectores lógicos, negación y agrupación, lo que obliga a resolver precedencia y asociatividad en la gramática, y (iii) las acciones admiten parámetros, lo que enriquece la sintaxis sin perder la propiedad LL(1).

Alcance. El lenguaje se llama **DomoDSL**. Se entregan: la especificación del lenguaje, la GLC y su análisis, FIRST/FOLLOW, la tabla LL(1), **dos** analizadores predictivos (descenso recursivo y dirigido por tabla), AST y árbol de derivación, mensajes de error localizados, 46 casos de prueba y 18 pruebas unitarias.

Estructura del informe. Sigue la organización sugerida en el enunciado: especificación del lenguaje (sección 2), gramática (3), análisis (4), FIRST y FOLLOW (5), tabla LL(1) (6), parser (7), árboles (8), errores (9), pruebas (10), conclusiones (11) y anexos (12).

2 Especificación del lenguaje

2.1 Propósito

DomoDSL permite escribir programas formados por una o más **reglas de automatización**. Cada regla evalúa una **condición** sobre variables del hogar (temperatura, humedad, movimiento, hora, ...) y, si se cumple, ordena una o más **acciones** sobre dispositivos (ventilador, alarma, termostato, ...). El lenguaje no ejecuta las reglas: el objetivo del proyecto es *reconocerlas* (decidir si son sintácticamente válidas), informar errores útiles y producir una representación estructurada (AST).

2.2 Alfabeto

El alfabeto relevante Σ está formado por:

- letras `A-Z` y `a-z`, dígitos `0-9`, el guion bajo `_` y el punto `.` (solo dentro de números decimales);
- los símbolos `>`, `<`, `=`, `!`, `;`, `,`, `(` y `)`;
- espacio, tabulación y salto de línea (separadores que el lexer descarta).

Cualquier otro carácter produce un *error léxico*.

2.3 Categorías de tokens

El analizador léxico (basado en el entregado por el docente) usa expresiones regulares. Los espacios se descartan. Las palabras reservadas **no distinguen mayúsculas de minúsculas** (como en la función base).

Token	Patrón	Descripción	Ejemplos
WHEN, THEN	WHEN | THEN	Inicio y separador de la regla	WHEN, then
AND, OR, NOT	AND | OR | NOT	Operadores lógicos	AND, not
TRUE, FALSE	TRUE | FALSE	Valores booleanos	TRUE
ID	[A-Za-z_][A-Za-z0-9_]*	Identificadores (variables, verbos y dispositivos)	temperatura, sensor_1
NUM	\d+(\.\d+)?	Números enteros o decimales	28, 0.5
OP_REL	>= | <= | == | != | > | <	Operadores relacionales	>=, ==
PUNTO_COMA	;	Fin de regla	;
COMA	,	Separador de acciones	,
PARENT_IZQ	(	Abre agrupación	(
PARENT_DER	)	Cierra agrupación	)

Un identificador cuyo texto coincide con una palabra reservada se convierte en el token reservado (p. ej. when → WHEN), por lo que las palabras reservadas no pueden usarse como identificadores. Al final de la entrada el lexer agrega el token $ (fin de entrada), que necesita el analizador LL(1).

2.4 Cumplimiento de los requisitos mínimos del enunciado

Requisito	Cómo lo cumple DomoDSL
Al menos 4 palabras reservadas	7: WHEN, THEN, AND, OR, NOT, TRUE, FALSE
Identificadores	Token ID
Valores numéricos y/o booleanos	NUM (enteros y decimales) y TRUE/FALSE
Al menos 3 operadores	9: seis relacionales (> < >= <= == !=) y tres lógicos (AND OR NOT)
Estructura de sentencia principal	WHEN <condición> THEN <acciones> ;
Más de una sentencia	Un programa es una secuencia de una o más reglas
Construcción compuesta	Conectores AND/OR, paréntesis y listas de acciones con parámetros
Extensión creativa	NOT, paréntesis, múltiples acciones y acciones con parámetros (sección 4.6)

2.5 Ejemplos de sentencias válidas e inválidas

Válidas.

1. `WHEN temperatura > 28 THEN encender ventilador;`
2. `WHEN movimiento == TRUE THEN activar alarma;`
3. `WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador;`
4. `WHEN luz < 100 OR hora >= 20 THEN encender lampara;`
5. `WHEN NOT puerta == TRUE THEN activar alarma;`
6. `WHEN (temperatura > 30 OR humedad > 80) AND presencia == TRUE THEN encender aire;`
7. `WHEN temperatura > 28 THEN encender ventilador 3, activar alarma;`

Inválidas.

N.º	Sentencia	Motivo
1	`WHEN temperatura > 28 encender ventilador;`	Falta la palabra reservada `THEN`
2	`WHEN temperatura > THEN encender ventilador;`	Falta el valor después de `>`
3	`temperatura > 28 THEN encender ventilador;`	La regla no comienza con `WHEN`
4	`WHEN temperatura > 28 THEN encender ventilador`	Falta el `;` final
5	`WHEN (temperatura > 28 THEN encender ventilador;`	Paréntesis sin cerrar
6	`WHEN temperatura = 28 THEN encender ventilador;`	`=` no es un operador (debe ser `==`): error léxico

3 Gramática libre de contexto

La gramática de DomoDSL es $G = (N, T, P, S)$ con:

- **No terminales** N (14): *programa*, *programa′*, *regla*, *condicion*, *condicion′*, *termino*, *termino′*, *factor*, *comparacion*, *valor*, *acciones*, *acciones′*, *accion*, *parametros*.
- **Terminales** T (14, son los tokens): `WHEN`, `THEN`, `PUNTO_COMA`, `OR`, `AND`, `NOT`, `PARENT_IZQ`, `PARENT_DER`, `ID`, `OP_REL`, `NUM`, `TRUE`, `FALSE`, `COMA`.
- **Símbolo inicial** S = *programa*.
- **Producciones** P: las 24 que muestra la tabla siguiente. (Las producciones numeradas son las que usa el código; el símbolo x' se escribe `x_p` en el programa.)

N.º	Producción
1	*programa* → *regla programa′*
2	*programa′* → *regla programa′*
3	*programa′* → ε
4	*regla* → `WHEN` *condicion* `THEN` *acciones* `PUNTO_COMA`
5	*condicion* → *termino condicion′*
6	*condicion′* → `OR` *termino condicion′*
7	*condicion′* → ε
8	*termino* → *factor termino′*
9	*termino′* → `AND` *factor termino′*
10	*termino′* → ε
11	*factor* → `NOT` *factor*
12	*factor* → `PARENT_IZQ` *condicion* `PARENT_DER`
13	*factor* → *comparacion*
14	*comparacion* → `ID` `OP_REL` *valor*
15	*valor* → `NUM`
16	*valor* → `TRUE`
17	*valor* → `FALSE`
18	*valor* → `ID`

N.º	Producción
19	*acciones* → *accion acciones′*
20	*acciones′* → `COMA` *accion acciones′*
21	*acciones′* → ε
22	*accion* → `ID ID` *parametros*
23	*parametros* → *valor parametros*
24	*parametros* → ε

Lectura de la gramática.

- Un **programa** es una o más reglas (1–3). Una **regla** es `WHEN condición THEN acciones ;` (4).
- Las **condiciones** se estratifican en tres niveles para codificar la precedencia: *condicion* (OR, la menor), *termino* (AND) y *factor* (NOT, paréntesis y comparaciones, la mayor) (5–13). Una **comparación** es `ID OP_REL valor` (14) y un **valor** es un número, un booleano o un identificador (15–18).
- Las **acciones** son una lista separada por comas (19–21); cada una es `verbo dispositivo [parámetros]` (22–24), donde los parámetros son cero o más valores.

Ejemplo de derivación. Derivación por la izquierda de `WHEN a > 1 THEN x y;` (los terminales se muestran como tipos de token: `ID`, `OP_REL`, `NUM`, ...). La derivación fue reconstruida a partir de la traza del analizador LL(1).

Paso	Prod.	Forma sentencial (derivación por la izquierda)
0		*programa*
1	1	⇒ *regla programa′*
2	4	⇒ `WHEN` *condicion* `THEN` *acciones* `PUNTO_COMA` *programa′*
3	5	⇒ `WHEN` *termino condicion′* `THEN` *acciones* `PUNTO_COMA` *programa′*
4	8	⇒ `WHEN` *factor termino′ condicion′* `THEN` *acciones* `PUNTO_COMA` *programa′*
5	13	⇒ `WHEN` *comparacion termino′ condicion′* `THEN` *acciones* `PUNTO_COMA` *programa′*
6	14	⇒ `WHEN ID OP_REL` *valor termino′ condicion′* `THEN` *acciones* `PUNTO_COMA` *programa′*
7	15	⇒ `WHEN ID OP_REL NUM` *termino′ condicion′* `THEN` *acciones* `PUNTO_COMA` *programa′*
8	10	⇒ `WHEN ID OP_REL NUM` *condicion′* `THEN` *acciones* `PUNTO_COMA` *programa′*
9	7	⇒ `WHEN ID OP_REL NUM THEN` *acciones* `PUNTO_COMA` *programa′*
10	19	⇒ `WHEN ID OP_REL NUM THEN` *accion acciones′* `PUNTO_COMA` *programa′*
11	22	⇒ `WHEN ID OP_REL NUM THEN ID ID` *parametros acciones′* `PUNTO_COMA` *programa′*
12	24	⇒ `WHEN ID OP_REL NUM THEN ID ID` *acciones′* `PUNTO_COMA` *programa′*
13	21	⇒ `WHEN ID OP_REL NUM THEN ID ID PUNTO_COMA` *programa′*
14	3	⇒ `WHEN ID OP_REL NUM THEN ID ID PUNTO_COMA`

4 Análisis de la gramática

4.1 Terminales, no terminales y símbolo inicial

Ya identificados en la sección 3: 14 terminales (los tokens), 14 no terminales y símbolo inicial *programa*. Los no terminales *programa'*, *condicion'*, *termino'*, *acciones'* son *auxiliares* introducidos al transformar la gramática;

el resto tiene significado propio del dominio.

4.2 Primera versión de la gramática (G_0)

Al escribir la gramática de forma natural se obtiene:

```
programa →regla programa | regla
regla →WHEN condicion THEN acciones PUNTO_COMA
condicion →condicion OR condicion | condicion AND condicion
         | NOT condicion | PARENT_IZQ condicion PARENT_DER | comparacion
comparacion →ID OP_REL valor
valor →NUM | TRUE | FALSE | ID
acciones →acciones COMA accion | accion
accion →ID ID parametros
parametros →parametros valor | ε
```

G_0 describe el lenguaje deseado, pero **no sirve para un analizador predictivo**: tiene recursión por la izquierda, prefijos comunes y es ambigua. A continuación se analiza cada problema.

4.3 Recursión por la izquierda

En G_0 hay recursión izquierda *directa* en tres no terminales: *condicion* (*condicion* → *condicion* `OR` ...), *acciones* y *parametros*. Un parser de descenso recursivo entraría en bucle infinito. Se aplica la transformación estándar $A \to A\alpha \mid \beta \implies A \to \beta A',\ A' \to \alpha A' \mid \varepsilon$:

Original (G_0)	**Transformado**	**En la gramática final**
acciones → *acciones* `COMA` *accion* | *accion*	*acciones* → *accion acciones′*	prods. 19–21
	acciones′ → `COMA` *accion acciones′* | ε	
parametros → *parametros valor* | ε	*parametros* → *valor parametros* | ε (recursión por la derecha, equivalente)	prods. 23–24
condicion → *condicion* `OR` *condicion* | ... (con el nivel OR separado: *condicion* → *condicion* `OR` *termino* | *termino*)	*condicion* → *termino condicion′*	prods. 5–7
	condicion′ → `OR` *termino condicion′* | ε	
(nivel AND, creado al separar la precedencia en la sección 4.5: *termino* → *termino* `AND` *factor* | *factor*)	*termino* → *factor termino′*	prods. 8–10
	termino′ → `AND` *factor termino′* | ε	

Las dos últimas filas parten de G_0 ya con los niveles de precedencia separados (*condicion* para OR y *termino* para AND), pues esa estratificación (sección 4.5) es la que elimina la ambigüedad; por eso *termino* no figura en G_0.

Verificación en la gramática final. No hay recursión izquierda directa (lo comprueba una prueba unitaria: ninguna producción empieza con su propio lado izquierdo). Tampoco indirecta: las dependencias por el primer símbolo forman cadenas sin ciclos:

- *programa* → *regla* → `WHEN`;
- *condicion* → *termino* → *factor* → *comparacion* → `ID`;
- *acciones* → *accion* → `ID`.

La única recursión de *factor* (*factor* → `NOT` *factor*) empieza con un terminal, por lo que es inofensiva.

4.4 Factorización por la izquierda

En G_0, *programa* tiene dos alternativas con prefijo común *regla* (*programa* → *regla programa* | *regla*): con un solo token de lookahead no se puede elegir. Se factoriza: *programa* → *regla programa'* con *programa'* → *programa* | ε; como *programa* empieza con *regla*, se sustituye y queda *programa'* → *regla programa'* | ε (prods. 1–3). En el resto de la gramática las alternativas de un mismo no terminal comienzan con terminales distintos (p. ej. *factor*: `NOT`, `PARENT_IZQ`, `ID`), por lo que **no se requiere más factorización**; esto se confirma formalmente en la sección 6 (conjuntos de selección disjuntos).

4.5 Ambigüedad

G_0 es **ambigua**: la cadena `a > 1 OR b > 2 AND c > 3` tiene dos árboles de derivación, según si la producción *condicion* → *condicion* `OR` *condicion* o *condicion* → *condicion* `AND` *condicion* se aplica en la raíz:

`(a > 1 OR b > 2) AND c > 3` frente a `a > 1 OR (b > 2 AND c > 3)`.

Además `a OR b OR c` puede agruparse a la izquierda o a la derecha (ambigüedad de asociatividad). Esto cambiaría el significado del programa, así que debe eliminarse en la gramática, no en el código.

Solución. Se separan los **niveles de precedencia** (*condicion* para OR, *termino* para AND, *factor* para NOT/paréntesis/comparación): AND liga más fuerte que OR, y NOT más que AND. Con las producciones 5–13 la cadena anterior tiene un único árbol, el de `a > 1 OR (b > 2 AND c > 3)`. La asociatividad es **izquierda**: el AST se construye en un ciclo que agrupa `((a OR b) OR c)` (sección 7). Como además la gramática final es LL(1) (sección 6), es **no ambigua** (toda gramática LL(1) lo es). La guía del enunciado (*condicion2* con AND/OR mezclados) no define precedencia entre ambos conectores; la estratificación usada aquí sí lo hace.

Otro punto revisado: en *accion*, los parámetros son identificadores/valores y podrían confundirse con el siguiente elemento; no hay ambigüedad porque la lista de parámetros termina siempre en `;` o `,` (FOLLOW(parametros) = {`PUNTO_COMA`, `COMA`}, ver sección 5).

4.6 Extensión creativa y su efecto en la gramática

La extensión de DomoDSL tiene tres componentes, todos integrados en la gramática, FIRST/FOLLOW, tabla, código y pruebas:

1. **Negación** `NOT`: *factor* → `NOT` *factor* (prod. 11); permite `NOT NOT x == 1`.
2. **Agrupación con paréntesis**: *factor* → `PARENT_IZQ` *condicion* `PARENT_DER` (prod. 12); es la que introduce `PARENT_DER` en FOLLOW(*condicion*) y justifica los mensajes "falta `)`" / "`)` sobrante".
3. **Acciones múltiples con parámetros** (propia del dominio): *acciones* con `COMA` (prods. 19–21) y *parametros* (prods. 23–24); p. ej. `ajustar termostato 22 TRUE, activar alarma`.

5 FIRST y FOLLOW

Definiciones. FIRST(α) es el conjunto de terminales que pueden iniciar una cadena derivada de α (incluye ε si $\alpha \Rightarrow^* \varepsilon$). FOLLOW($A$) es el conjunto de terminales que pueden aparecer inmediatamente a la derecha de A en alguna forma sentencial (incluye `$` si A puede ir al final).

Reglas usadas.

1. Si X es terminal, $\text{FIRST}(X) = \{X\}$. Si $X \to \varepsilon$, $\varepsilon \in \text{FIRST}(X)$.
2. Si $X \to Y_1 Y_2 \cdots Y_k$: se agrega $\text{FIRST}(Y_1) \setminus \{\varepsilon\}$; si Y_1 es anulable, también $\text{FIRST}(Y_2) \setminus \{\varepsilon\}$, y así sucesivamente; $\varepsilon \in \text{FIRST}(X)$ si todos los Y_i son anulables.
3. \$∈ $\text{FOLLOW}(S)$.
4. Si $A \to \alpha B \beta$: $\text{FIRST}(\beta) \setminus \{\varepsilon\} \subseteq \text{FOLLOW}(B)$.
5. Si $A \to \alpha B$ o $A \to \alpha B \beta$ con $\varepsilon \in \text{FIRST}(\beta)$: $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$.

Los conjuntos se calculan iterando hasta alcanzar el punto fijo (`calcular_first` y `calcular_follow` en `gramatica.py`); los resultados coinciden con el cálculo manual justificado en las tablas y están verificados por pruebas unitarias (`test_first`, `test_follow`).

5.1 Conjuntos FIRST

No terminal	FIRST	Justificación
programa	{ `WHEN` }	Prod. 1: empieza por *regla*, que no deriva ε; FIRST(programa) = FIRST(regla).
programa′	{ `WHEN`, ε }	Prod. 2: FIRST(regla) = {WHEN}; prod. 3 aporta ε.
regla	{ `WHEN` }	Prod. 4: empieza con el terminal WHEN.
condicion	{ `NOT`, `PARENT_IZQ`, `ID` }	Prod. 5: *termino* no es anulable, luego FIRST(condicion) = FIRST(termino).
condicion′	{ `OR`, ε }	Prod. 6 empieza con OR; prod. 7 aporta ε.
termino	{ `NOT`, `PARENT_IZQ`, `ID` }	Prod. 8: *factor* no es anulable, luego FIRST(termino) = FIRST(factor).
termino′	{ `AND`, ε }	Prod. 9 empieza con AND; prod. 10 aporta ε.
factor	{ `NOT`, `PARENT_IZQ`, `ID` }	Prod. 11: NOT; prod. 12: PARENT_IZQ; prod. 13: FIRST(comparacion) = {ID}.
comparacion	{ `ID` }	Prod. 14: empieza con el terminal ID.
valor	{ `ID`, `NUM`, `TRUE`, `FALSE` }	Prods. 15–18: NUM, TRUE, FALSE e ID.
acciones	{ `ID` }	Prod. 19: FIRST(accion) = {ID}; *accion* no es anulable.
acciones′	{ `COMA`, ε }	Prod. 20 empieza con COMA; prod. 21 aporta ε.
accion	{ `ID` }	Prod. 22: empieza con el terminal ID.
parametros	{ `ID`, `NUM`, `TRUE`, `FALSE`, ε }	Prod. 23: FIRST(valor); prod. 24 aporta ε.

Los no terminales anulables son *programa'*, *condicion'*, *termino'*, *acciones'* y *parametros*.

5.2 Conjuntos FOLLOW

No terminal	FOLLOW	Justificación
programa	{ `$` }	Símbolo inicial: se agrega \$.
programa′	{ `$` }	Prod. 1: *programa'* está al final, luego FOLLOW(programa) ⊆ FOLLOW(programa'); la prod. 2 solo repite la regla.

No terminal	FOLLOW	Justificación
regla	{ WHEN, $ }	Prods. 1 y 2: sigue *programa'*; FIRST(programa')\{ε} = {WHEN} y como es anulable se agrega FOLLOW(programa') = {$}.
condicion	{ THEN, PARENT_DER }	Prod. 4: sigue THEN. Prod. 12: sigue PARENT_DER.
condicion'	{ THEN, PARENT_DER }	Prods. 5 y 6: va al final, luego hereda FOLLOW(condicion).
termino	{ THEN, OR, PARENT_DER }	Prods. 5 y 6: sigue *condicion'*; FIRST = {OR} y es anulable, luego se agrega FOLLOW(condicion).
termino'	{ THEN, OR, PARENT_DER }	Prods. 8 y 9: va al final, luego hereda FOLLOW(termino).
factor	{ THEN, OR, AND, PARENT_DER }	Prods. 8 y 9: sigue *termino'*: AND, y por ser anulable FOLLOW(termino). Prod. 11 (NOT factor): hereda FOLLOW(factor) (sin efecto nuevo).
comparacion	{ THEN, OR, AND, PARENT_DER }	Prod. 13: va al final, luego hereda FOLLOW(factor).
valor	{ THEN, PUNTO_COMA, OR, AND, PARENT_DER, ID, NUM, TRUE, FALSE, COMA }	Prod. 14: va al final, hereda FOLLOW(comparacion). Prod. 23: sigue *parametros*: FIRST(parametros)\{ε} y, al ser anulable, FOLLOW(parametros).
acciones	{ PUNTO_COMA }	Prod. 4: sigue PUNTO_COMA.
acciones'	{ PUNTO_COMA }	Prods. 19 y 20: va al final, luego hereda FOLLOW(acciones).
accion	{ PUNTO_COMA, COMA }	Prods. 19 y 20: sigue *acciones'*: COMA, y por ser anulable FOLLOW(acciones) (y FOLLOW(acciones')).
parametros	{ PUNTO_COMA, COMA }	Prod. 22: va al final, hereda FOLLOW(accion). Prod. 23: va al final, hereda FOLLOW(parametros).

6 Tabla LL(1)

Construcción. Para cada producción $A \to \alpha$ (numerada n): (a) para cada terminal $a \in \text{FIRST}(\alpha) \setminus \{\varepsilon\}$, se escribe n en $M[A, a]$; (b) si $\varepsilon \in \text{FIRST}(\alpha)$, se escribe n en $M[A, b]$ para cada $b \in \text{FOLLOW}(A)$ (incluido $). Una celda vacía significa error sintáctico.

Cuadro 5: Tabla predictiva LL(1) de DomoDSL (cada número es la producción de la sección 3; celda vacía = error).

	WHEN	THEN	PUNTO_COMA	OR	AND	NOT	PARENT_IZQ	PARENT_DER	ID	OP_REL	NUM	TRUE	FALSE	COMA	$
programa	**1**														
programa′	**2**														**3**
regla	**4**														
condicion						**5**	**5**		**5**						
condicion′		**7**		**6**				**7**							
termino						**8**	**8**		**8**						
termino′		**10**		**10**	**9**			**10**							
factor						**11**	**12**		**13**						
comparacion									**14**						
valor									**18**		**15**	**16**	**17**		
acciones									**19**						
acciones′			**21**											**20**	
accion									**22**						
parametros			**24**						**23**		**23**	**23**	**23**	**24**	

6.1 ¿Es la gramática LL(1)?

Sí. Ninguna celda contiene más de una producción (el programa lo verifica: `CONFLICTOS = []`). Equivalentemente, para cada no terminal con varias alternativas los *conjuntos de selección* $\text{SEL}(A \to \alpha) = \text{FIRST}(\alpha) \setminus \{\varepsilon\} \cup (\text{FOLLOW}(A) \text{ si } \varepsilon \in \text{FIRST}(\alpha))$ son disjuntos de a pares:

No terminal	Producción	Conjunto de selección	¿Disjuntos?
programa′	(2) *regla programa′*	{ WHEN }	**sí**
	(3) ε	{ $ }	
condicion′	(6) OR *termino condicion′*	{ OR }	**sí**
	(7) ε	{ THEN, PARENT_DER }	
termino′	(9) AND *factor termino′*	{ AND }	**sí**
	(10) ε	{ THEN, OR, PARENT_DER }	
factor	(11) NOT *factor*	{ NOT }	**sí**
	(12) PARENT_IZQ *condicion* PARENT_DER	{ PARENT_IZQ }	
	(13) *comparacion*	{ ID }	
valor	(15) NUM	{ NUM }	**sí**
	(16) TRUE	{ TRUE }	
	(17) FALSE	{ FALSE }	
	(18) ID	{ ID }	
acciones′	(20) COMA *accion acciones′*	{ COMA }	**sí**
	(21) ε	{ PUNTO_COMA }	

No terminal	Producción	Conjunto de selección	¿Disjuntos?
parametros	(23) *valor parametros*	{ `ID`, `NUM`, `TRUE`, `FALSE` }	**sí**
	(24) ε	{ `PUNTO_COMA`, `COMA` }	

Puntos delicados y por qué no hay conflicto.

- *parametros*: las producciones *valor parametros* (selección {`NUM`, `TRUE`, `FALSE`, `ID`}) y ε (selección FOLLOW = {`PUNTO_COMA`, `COMA`}) no se solapan, precisamente porque la lista de parámetros siempre termina en `;` o `,`.
- *termino'* y *condicion'*: la alternativa vacía se elige con FOLLOW (`OR`, `THEN`, `)`), que no contiene `AND`; por eso AND se asocia siempre antes que OR (precedencia).
- *programa'*: `WHEN` inicia otra regla y `$` termina el programa.

Conclusión. No fue necesario modificar la gramática después de calcular la tabla: las transformaciones de la sección 4 ya dejaron una gramática LL(1) utilizable por ambos analizadores.

7 Diseño e implementación del parser

7.1 Arquitectura

Módulo	Responsabilidad
`lexer.py`	Texto → lista de `Token` (tipo, lexema, posición, línea, columna) terminada en `$`. Error léxico para símbolos desconocidos.
`gramatica.py`	Lista `PRODUCCIONES`; `calcular_first`, `calcular_follow`, `construir_tabla`; detección de conflictos.
`parser_rd.py`	Parser predictivo por **descenso recursivo**; construye el **AST**.
`parser_ll1.py`	Parser **dirigido por la tabla LL(1)** con pila; construye el árbol de derivación y una traza.
`arbol.py`	Clases del AST, árbol genérico, texto jerárquico y exportación a Graphviz.
`errores.py`	`ErrorLexico` y `ErrorSintactico` (mensaje, posición, línea, columna, puntero ^).
`main.py`	Interfaz de línea de comandos (archivo, texto, modo interactivo, opciones de visualización).
`casos_prueba.py`, `ejecutar_pruebas.py`, `test_parser.py`	Casos, ejecutor con ambos parsers y pruebas unitarias.

Ningún módulo usa bibliotecas de generación de parsers: solo la biblioteca estándar de Python (`re`, `dataclasses`, `argparse`, ...).

7.2 Analizador léxico

Parte de la función `analizador_lexico` entregada por el docente (expresiones regulares nombradas combinadas en un patrón único). Se ampliaron: el token `COMA`; números decimales; el **registro de posición** (offset, línea y columna) de cada token, necesario para ubicar los errores; el token de fin `$`; y mensajes léxicos con sugerencia ("¿quiso escribir ==?").

7.3 Parser por descenso recursivo

Hay **un método por cada no terminal principal** (*programa*, *regla*, *condicion*, *termino*, *factor*, *comparacion*, *valor*, *acciones* y *accion*), y cada método implementa sus producciones; los comentarios del código indican el número de producción. Los no terminales auxiliares (*programa'*, *condicion'*, *termino'*, *acciones'*, *parametros*) se implementan como ciclos `while`, que equivalen a la recursión por la derecha de la gramática pero (i) no desbordan la pila de Python con programas largos y (ii) permiten construir el AST con asociatividad izquierda. Cuando la alternativa elegida es ε, el parser comprueba el lookahead contra FOLLOW de forma explícita en *termino'* (conjunto `FOLLOW["termino_p"]`) y en *programa'* (solo `$`); en *condicion'*, *acciones'* y *parametros* la comprobación ocurre al exigir el terminal siguiente (`THEN` o `;`). Así el error se informa en el punto exacto en que aparece.

Producciones	Método (`ParserDescendente`)
1–3 (*programa*, *programa'*)	`programa()`
4 (*regla*)	`regla()`
5–7 (*condicion*, *condicion'*)	`condicion()` — ciclo `while OR`
8–10 (*termino*, *termino'*)	`termino()` — ciclo `while AND` + verificación FOLLOW
11–13 (*factor*)	`factor()`
14 (*comparacion*)	`comparacion()`
15–18 (*valor*)	`valor()`
19–21 (*acciones*, *acciones'*)	`acciones()` — ciclo `while COMA`
22–24 (*accion*, *parametros*)	`accion()` — ciclo `while` lookahead $\in$ FIRST(valor)

```
105     def condicion(self, contexto):
106        izq = self.termino(contexto) # condicion →termino condicion_p
107        while self.actual.tipo == "OR": # condicion_p →OR termino condicion_p
108           self.avanzar()
109           der = self.termino("después de OR")
110           izq = Or(izq, der) # asociatividad izquierda
111        return izq # condicion_p →ε(lookahead ∈FOLLOW)
112
113     # ---------------------------------------------------- <termino> (8,9,10)
114     def termino(self, contexto):
115        izq = self.factor(contexto) # termino →factor termino_p
116        while self.actual.tipo == "AND": # termino_p →AND factor termino_p
117           self.avanzar()
118           der = self.factor("después de AND")
119           izq = And(izq, der)
120        # termino_p →ε: el lookahead debe estar en FOLLOW(termino_p)
121        if self.actual.tipo not in FOLLOW["termino_p"]:
122           if self.profundidad > 0:
123              self.error_esperaba("')' para cerrar el paréntesis abierto "
124                                  f"en la columna {self.parentesis_abiertos[-1].columna} "
125                                  "(o un conector AND / OR)")
126           self.error_esperaba("la palabra reservada THEN (o un conector AND / OR)",
127                               "después de la condición")
128        if self.actual.tipo == "PARENT_DER" and self.profundidad == 0:
129           self.error("paréntesis de cierre ')' sin su '(' correspondiente.")
130        return izq
131
132     # ---------------------------------------------------- <factor> (11,12,13)
133     def factor(self, contexto):
134        if self.actual.tipo == "NOT": # factor →NOT factor
135           self.avanzar()
136           return Not(self.factor("después de NOT"))
137        if self.actual.tipo == "PARENT_IZQ": # factor →( condicion )
138           abre = self.avanzar()
```

```
        self.profundidad += 1
        self.parentesis_abiertos.append(abre)
        interno = self.condicion("después de '('")
        if self.actual.tipo != "PARENT_DER":
            self.error_esperaba(f"')' para cerrar el paréntesis abierto en la columna "
                                f"{abre.columna}")
        self.avanzar()
        self.parentesis_abiertos.pop()
        self.profundidad -= 1
        return interno
    return self.comparacion(contexto) # factor →comparacion

# ------------------------------------------------ <comparacion> (14)
```

Listing 1: Extracto de parser_rd.py: condicion, termino y factor (niveles de precedencia).

7.4 Parser dirigido por tabla LL(1)

El segundo analizador usa la *misma* lista de producciones: la tabla se genera automáticamente desde `gramatica.py` y se interpreta con una pila (inicializada con `$ programa`). Si el tope es un terminal, debe coincidir con el token actual; si es un no terminal, se consulta $M[X, a]$, se reemplaza X por el lado derecho de la producción (en orden inverso) y se agregan los nodos hijos al árbol de derivación. Una celda vacía produce un error que lista los terminales esperados (la fila de la tabla). La opción `-traza` imprime cada paso (pila, entrada restante, acción); las capturas de la ejecución están en el Anexo B.

```
while pila:
    simbolo, nodo = pila[-1]
    token = tokens[i]

    if traza is not None:
        restante = " ".join(t.tipo for t in tokens[i:])
        contenido = " ".join(s for s, _ in reversed(pila))
        accion = ""

    if not es_no_terminal(simbolo): # --- terminal ---
        if simbolo != token.tipo:
            raise ErrorSintactico(
                f"se esperaba {NOMBRES_AMIGABLES[simbolo]}, "
                f"pero se encontró {describir(token)}.",
                token.pos, token.linea, token.columna)
        pila.pop()
        nodo.etiqueta = f"{simbolo} '{token.lexema}'" if token.lexema else simbolo
        if traza is not None:
            traza.append((contenido, restante, f"coincidir {simbolo}"))
        if simbolo != FIN:
            i += 1
        continue

    nprod = TABLA[simbolo].get(token.tipo) # --- no terminal ---
    if nprod is None:
        raise ErrorSintactico(
            f"símbolo inesperado: se encontró {describir(token)}; "
            f"en este punto (<{simbolo}>) se esperaba {_lista_legible(_esperados(simbolo))}.",
            token.pos, token.linea, token.columna)
    pila.pop()
    _, rhs = PRODUCCIONES[nprod - 1]
    hijos = []
    for s in rhs:
        hijos.append(Nodo(EPS, terminal=True) if s == EPS
                     else Nodo(s, terminal=not es_no_terminal(s)))
    nodo.hijos = hijos
```

```
83        for s, h in reversed(list(zip(rhs, hijos))):
84           if s != EPS:
85              pila.append((s, h))
86        if traza is not None:
87           traza.append((contenido, restante, f"({nprod}) {formatear_produccion(nprod)}"))
```

Listing 2: Extracto de parser_ll1.py: bucle principal del analizador LL(1).

7.5 Coherencia entre gramática, informe y código

La lista `PRODUCCIONES` es la *única* fuente de verdad: de ella salen FIRST, FOLLOW, la tabla, el parser LL(1) y las tablas de este informe, que coinciden con la salida de `gramatica.py` (comprobado automáticamente: producciones, FIRST, FOLLOW y tabla). El parser por descenso recursivo se escribió a mano siguiendo cada producción, y la suite de pruebas comprueba que ambos analizadores aceptan y rechazan exactamente las mismas entradas (46 casos).

8 Árbol sintáctico / AST

El sistema construye dos representaciones:

- **Árbol de derivación** (concreto), generado por el parser LL(1): contiene todos los no terminales, incluidos los auxiliares y las hojas ε.
- **AST** (abstracto), generado por el parser de descenso recursivo: elimina los no terminales auxiliares, los paréntesis y las hojas ε; los operadores son nodos internos y la asociatividad izquierda queda explícita (`Or(Or(a,b),c)`).

Ambos pueden mostrarse como texto jerárquico o exportarse a Graphviz (`-dot`).

8.1 AST de la regla del enunciado

Entrada: `WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador;`

```
Programa
‘-- Regla (línea 1)
    |-- Condición
    |   ‘-- AND
    |       |-- >
    |       |   |-- temperatura  [ID]
    |       |   ‘-- 28  [NUM]
    |       ‘-- <
    |           |-- humedad  [ID]
    |           ‘-- 40  [NUM]
    ‘-- Acciones
        ‘-- Acción: encender
            ‘-- objeto: ventilador
```

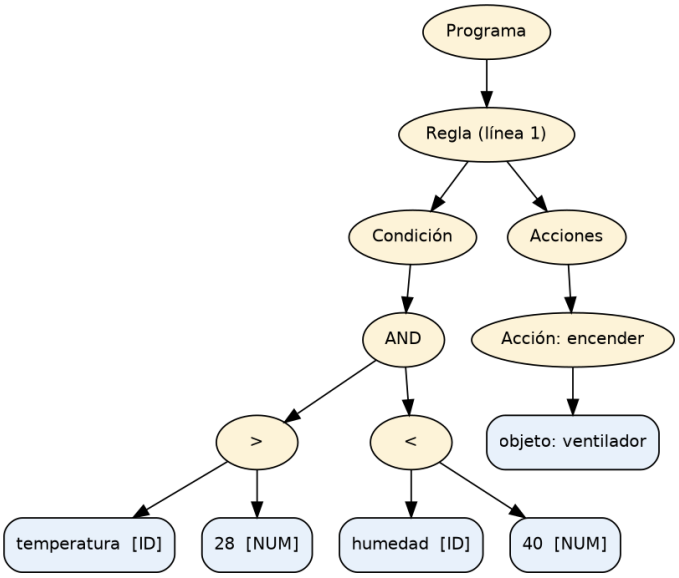


Figura 1: AST generado (Graphviz) para la regla del enunciado. Óvalos: nodos internos; cajas: hojas.

8.2 AST con la extensión (NOT, paréntesis, acciones múltiples con parámetros)

Entrada:

```
WHEN NOT (puerta == TRUE) AND (hora >= 22 OR hora < 6)
THEN bloquear puerta_principal, ajustar termostato 22;
```

Se observa que el AST refleja la precedencia (`AND` es la raíz de la condición; `OR` queda dentro del segundo operando por los paréntesis) y que los paréntesis no aparecen como nodos: su efecto está en la forma del árbol.

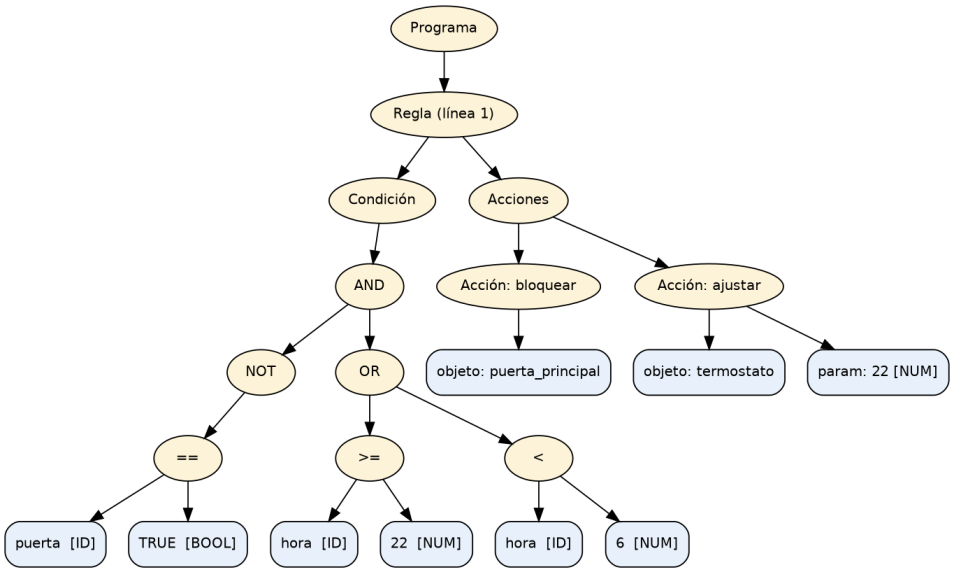


Figura 2: AST de una regla con `NOT`, paréntesis, dos acciones y un parámetro.

8.3 Árbol de derivación (parser LL(1))

Para `WHEN a > 1 THEN x y;;` las hojas ε corresponden a las producciones vacías 3, 7, 10, 21 y 24.

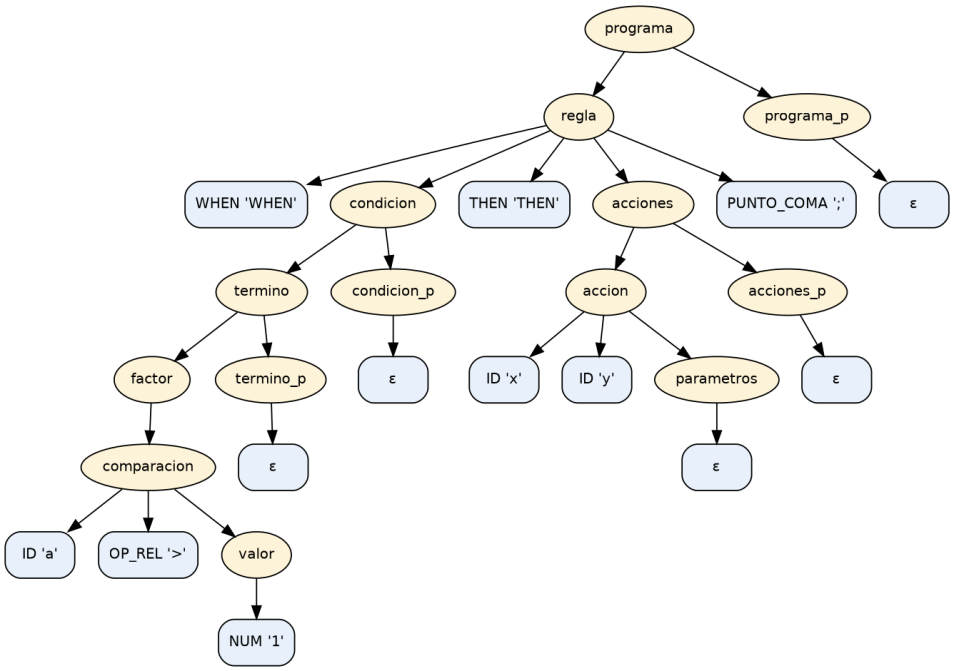


Figura 3: Árbol de derivación obtenido con la tabla LL(1).

9 Manejo de errores

Estrategia. El analizador se detiene en el **primer error** y lo informa con tres datos de ubicación (línea, columna y posición/offset), el mensaje, la línea de código afectada y un puntero ^ bajo el token problemático. Hay dos clases de error: `ErrorLexico` (carácter fuera del alfabeto) y `ErrorSintactico`. Los mensajes siguen el formato *se esperaba X [contexto], pero se encontró Y*, y dependen del contexto sintáctico:

- Después de la condición se pide `THEN` (o un conector AND/OR) y, dentro de un paréntesis, se pide `)` indicando la columna donde se abrió.
- Un `)` sin `(` se informa como "paréntesis de cierre sin su `(` correspondiente".
- Cada punto de la gramática tiene su mensaje: valor tras el operador, operador tras el identificador, dispositivo tras la acción, `;` al final, etc.
- La entrada vacía tiene un mensaje propio.

En este informe los mensajes se abrevian como `Error sintáctico [l:c]: ...`, donde l es la línea y c la columna. El programa imprime el mismo texto con el formato `Error sintáctico (línea` l`, columna` c`, posición` p`): ...`, seguido de la línea afectada y el puntero ^, donde p es el *offset* (base 0).

9.1 Mensajes del analizador de descenso recursivo

Caso	Entrada	Mensaje entregado por el analizador
I01	`WHEN temperatura > 28 encender ventilador;`	Error sintáctico [1:23]: se esperaba la palabra reservada THEN (o un conector AND / OR) después de la condición, pero se encontró el identificador 'encender'.
I02	`WHEN temperatura > THEN encender ventilador;`	Error sintáctico [1:20]: se esperaba un valor (número, TRUE, FALSE o identificador) después del operador >, pero se encontró la palabra reservada THEN.

Caso	Entrada	Mensaje entregado por el analizador
I03	`WHEN temperatura 28 THEN encender ventilador;`	Error sintáctico [1:18]: se esperaba un operador relacional ('>', '<', '>=', '<=', '==', '!=') después de 'temperatura', pero se encontró el número 28.
I05	`WHEN temperatura > 28 THEN encender ventilador`	Error sintáctico [1:47]: se esperaba ';' al final de la regla (o ',' para encadenar otra acción), pero se encontró el fin de la entrada.
I06	`WHEN temperatura > 28 THEN ;`	Error sintáctico [1:28]: se esperaba el nombre de una acción (identificador) después de THEN, pero se encontró el símbolo ';'.
I07	`WHEN temperatura > 28 THEN encender;`	Error sintáctico [1:36]: se esperaba el dispositivo u objeto (identificador) después de la acción 'encender', pero se encontró el símbolo ';'.
I08	`WHEN (temperatura > 28 THEN encender ventilador;`	Error sintáctico [1:24]: se esperaba ')' para cerrar el paréntesis abierto en la columna 6, pero se encontró la palabra reservada THEN.
I09	`WHEN temperatura > 28) THEN encender ventilador;`	Error sintáctico [1:22]: paréntesis de cierre ')' sin su '(' correspondiente.
I10	`WHEN temperatura > 28 AND THEN encender ventilador;`	Error sintáctico [1:27]: se esperaba una condición (un identificador, NOT o '(') después de AND, pero se encontró la palabra reservada THEN.
I15	`WHEN temperatura > 28 THEN encender ventilador @;`	Error léxico [1:48]: símbolo no reconocido '@'.

9.2 Mensajes del analizador dirigido por tabla

El parser LL(1) usa un mensaje más genérico, construido a partir de la fila de la tabla (los terminales para los que existe producción):

Caso	Entrada	Mensaje del analizador dirigido por tabla
I01	`WHEN temperatura > 28 encender ventilador;`	Error sintáctico [1:23]: símbolo inesperado: se encontró el identificador 'encender'; en este punto (<termino_p>) se esperaba la palabra reservada THEN, AND, OR o ')'.
I02	`WHEN temperatura > THEN encender ventilador;`	Error sintáctico [1:20]: símbolo inesperado: se encontró la palabra reservada THEN; en este punto (<valor>) se esperaba TRUE, FALSE, un identificador o un número.
I10	`WHEN temperatura > 28 AND THEN encender ventilador;`	Error sintáctico [1:27]: símbolo inesperado: se encontró la palabra reservada THEN; en este punto (<factor>) se esperaba NOT, un identificador o '('.

Ambos analizadores coinciden siempre en *aceptar o rechazar*; el de descenso recursivo da mensajes más específicos porque conoce el contexto (el motivo por el que se llegó a cada método). Las capturas con el formato real, con puntero, están en el Anexo B.

Limitación. No hay recuperación de errores (modo pánico): tras el primer error el análisis termina. Sería una extensión natural sincronizar en `;` (que pertenece a FOLLOW de *acciones*) para informar varios errores por ejecución.

10 Casos de prueba

Se documentan **46 casos**: 12 válidas, 20 inválidas y 14 casos límite (el enunciado exige 10, 10 y 5; la rúbrica premia 25 o más pruebas variadas). Cada caso se ejecutó con **ambos analizadores**; el resultado de las tablas es el obtenido por el programa (`python ejecutar_pruebas.py`). Resultado global: **46/46 casos correctos**. Además hay 18 pruebas unitarias (`python -m unittest`) que verifican el léxico, FIRST, FOLLOW, la ausencia de conflictos, la precedencia y asociatividad del AST y la posición de los errores. Los casos de entradas muy largas (L05–L07) se muestran abreviados; su texto completo se genera en `casos_prueba.py`.

10.1 Entradas sintácticamente válidas

ID	Entrada	Resultado esperado	Resultado obtenido	Observación
V01	`WHEN temperatura > 28 THEN encender ventilador;`	ACEPTA	ACEPTA ✓	Regla mínima del enunciado
V02	`WHEN movimiento == TRUE THEN activar alarma;`	ACEPTA	ACEPTA ✓	Valor booleano
V03	`WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador;`	ACEPTA	ACEPTA ✓	Conector AND
V04	`WHEN luz < 100 OR hora >= 20 THEN encender lampara;`	ACEPTA	ACEPTA ✓	Conector OR y operador >=
V05	`WHEN NOT puerta == TRUE THEN activar alarma;`	ACEPTA	ACEPTA ✓	Extensión: NOT
V06	`WHEN (temperatura > 30 OR humedad > 80) AND presencia == TRUE THEN encender aire;`	ACEPTA	ACEPTA ✓	Extensión: paréntesis (agrupación)
V07	`WHEN temperatura > 28 THEN encender ventilador 3, activar alarma;`	ACEPTA	ACEPTA ✓	Extensión: múltiples acciones y parámetro
V08	`WHEN temperatura >= 26.5 THEN ajustar termostato 22 TRUE;`	ACEPTA	ACEPTA ✓	Número decimal y varios parámetros
V09	`WHEN temperatura > 28 THEN encender ventilador;\nWHEN movimiento == TRUE THEN activar alarma;`	ACEPTA	ACEPTA ✓	Dos reglas en líneas distintas
V10	`WHEN a != b THEN x y;`	ACEPTA	ACEPTA ✓	Comparación ID contra ID
V11	`WHEN NOT NOT a == 1 THEN x y;`	ACEPTA	ACEPTA ✓	NOT anidado
V12	`WHEN a > 1 AND (b < 2 OR NOT (c == TRUE)) THEN x y 1 2 3 TRUE FALSE z;`	ACEPTA	ACEPTA ✓	Combina NOT, paréntesis y lista larga de parámetros

10.2 Entradas sintácticamente inválidas

ID	Entrada	Resultado esperado	Resultado obtenido	Observación
I01	`WHEN temperatura > 28 encender ventilador;`	RECHAZA (el mensaje menciona «la palabra reservada THEN»)	RECHAZA ✓ Error sintáctico [1:23]: se esperaba la palabra reservada THEN (o un conector AND / OR) después de la condición, pero se encontró el identificador 'encender'.	Falta THEN
I02	`WHEN temperatura > THEN encender ventilador;`	RECHAZA (el mensaje menciona «después del operador >»)	RECHAZA ✓ Error sintáctico [1:20]: se esperaba un valor (número, TRUE, FALSE o identificador) después del operador >, pero se encontró la palabra reservada THEN.	Falta el valor tras el operador
I03	`WHEN temperatura 28 THEN encender ventilador;`	RECHAZA (el mensaje menciona «después de 'temperatura'»)	RECHAZA ✓ Error sintáctico [1:18]: se esperaba un operador relacional ('>', '<', '>=', '<=', '==', '!=') después de 'temperatura', pero se encontró el número 28.	Falta el operador relacional
I04	`temperatura > 28 THEN encender ventilador;`	RECHAZA (el mensaje menciona «la palabra reservada WHEN»)	RECHAZA ✓ Error sintáctico [1:1]: se esperaba la palabra reservada WHEN al inicio de una regla, pero se encontró el identificador 'temperatura'.	La regla no comienza con WHEN
I05	`WHEN temperatura > 28 THEN encender ventilador`	RECHAZA (el mensaje menciona «';'»)	RECHAZA ✓ Error sintáctico [1:47]: se esperaba ';' al final de la regla (o ',' para encadenar otra acción), pero se encontró el fin de la entrada.	Falta el punto y coma final
I06	`WHEN temperatura > 28 THEN ;`	RECHAZA (el mensaje menciona «el nombre de una acción»)	RECHAZA ✓ Error sintáctico [1:28]: se esperaba el nombre de una acción (identificador) después de THEN, pero se encontró el símbolo ';'.	Regla sin acción
I07	`WHEN temperatura > 28 THEN encender;`	RECHAZA (el mensaje menciona «después de la acción 'encender'»)	RECHAZA ✓ Error sintáctico [1:36]: se esperaba el dispositivo u objeto (identificador) después de la acción 'encender', pero se encontró el símbolo ';'.	Acción sin dispositivo
I08	`WHEN (temperatura > 28 THEN encender ventilador;`	RECHAZA (el mensaje menciona «')' para cerrar el paréntesis»)	RECHAZA ✓ Error sintáctico [1:24]: se esperaba ')' para cerrar el paréntesis abierto en la columna 6, pero se encontró la palabra reservada THEN.	Paréntesis sin cerrar
I09	`WHEN temperatura > 28) THEN encender ventilador;`	RECHAZA (el mensaje menciona «sin su '(' correspondiente»)	RECHAZA ✓ Error sintáctico [1:22]: paréntesis de cierre ')' sin su '(' correspondiente.	Paréntesis de cierre sobrante
I10	`WHEN temperatura > 28 AND THEN encender ventilador;`	RECHAZA (el mensaje menciona «después de AND»)	RECHAZA ✓ Error sintáctico [1:27]: se esperaba una condición (un identificador, NOT o '(') después de AND, pero se encontró la palabra reservada THEN.	AND sin segundo operando
I11	`WHEN temperatura > 28 OR OR humedad < 3 THEN x y;`	RECHAZA (el mensaje menciona «después de OR»)	RECHAZA ✓ Error sintáctico [1:26]: se esperaba una condición (un identificador, NOT o '(') después de OR, pero se encontró la palabra reservada OR.	Dos OR seguidos

ID	Entrada	Resultado esperado	Resultado obtenido	Observación
I12	`WHEN temperatura = 28 THEN encender ventilador;`	RECHAZA (el mensaje menciona «¿quiso escribir '=='»)	RECHAZA ✓ Error léxico [1:18]: símbolo no reconocido '='. ¿quiso escribir '==' para comparar?	Error léxico: '=' en vez de '=='
I13	`WHEN temperatura > 28 THEN encender ventilador;\nWHEN humedad < 40 THEN encender bomba`	RECHAZA (el mensaje menciona «línea 2»)	RECHAZA ✓ Error sintáctico [2:38]: se esperaba ';' al final de la regla (o ',' para encadenar otra acción), pero se encontró el fin de la entrada.	Segunda regla sin ';' (error localizado en la línea 2)
I14	`WHEN 28 > temperatura THEN encender ventilador;`	RECHAZA (el mensaje menciona «el número 28»)	RECHAZA ✓ Error sintáctico [1:6]: se esperaba una condición (un identificador, NOT o '(') después de WHEN, pero se encontró el número 28.	Un número a la izquierda del operador
I15	`WHEN temperatura > 28 THEN encender ventilador @;`	RECHAZA (el mensaje menciona «símbolo no reconocido '@'»)	RECHAZA ✓ Error léxico [1:48]: símbolo no reconocido '@'.	Error léxico: carácter no válido
I16	`WHEN temperatura > 28 THEN encender ventilador;;`	RECHAZA (el mensaje menciona «una nueva regla (WHEN) o el fin de la entrada»)	RECHAZA ✓ Error sintáctico [1:48]: se esperaba una nueva regla (WHEN) o el fin de la entrada después de ';', pero se encontró el símbolo ';'.	Punto y coma duplicado
I17	`WHEN temperatura > 28 THEN encender ventilador, ;`	RECHAZA (el mensaje menciona «después de ','»)	RECHAZA ✓ Error sintáctico [1:49]: se esperaba el nombre de una acción (identificador) después de ',', pero se encontró el símbolo ';'.	Coma sin acción posterior
I18	`WHEN NOT THEN x y;`	RECHAZA (el mensaje menciona «después de NOT»)	RECHAZA ✓ Error sintáctico [1:10]: se esperaba una condición (un identificador, NOT o '(') después de NOT, pero se encontró la palabra reservada THEN.	NOT sin operando
I19	`WHEN a > 1 THEN x y WHEN b < 3 THEN z w;`	RECHAZA (el mensaje menciona «la palabra reservada WHEN»)	RECHAZA ✓ Error sintáctico [1:21]: se esperaba ';' al final de la regla (o ',' para encadenar otra acción), pero se encontró la palabra reservada WHEN.	Falta ';' entre dos reglas
I20	`WHEN a > 1 THEN 5 ventilador;`	RECHAZA (el mensaje menciona «el nombre de una acción»)	RECHAZA ✓ Error sintáctico [1:17]: se esperaba el nombre de una acción (identificador) después de THEN, pero se encontró el número 5.	La acción debe ser un identificador

10.3 Casos límite

ID	Entrada	Resultado esperado	Resultado obtenido	Observación
L01	`(cadena vacía)`	RECHAZA (el mensaje menciona «la entrada está vacía»)	RECHAZA ✓ Error sintáctico [1:1]: la entrada está vacía: se esperaba al menos una regla que comience con la palabra reservada WHEN.	Entrada vacía

ID	Entrada	Resultado esperado	Resultado obtenido	Observación
L02	`(espacios, tabulaciones y saltos de línea)`	RECHAZA (el mensaje menciona «la entrada está vacía»)	RECHAZA ✓ Error sintáctico [3:1]: la entrada está vacía: se esperaba al menos una regla que comience con la palabra reservada WHEN.	Solo espacios y saltos de línea
L03	`WHEN`	RECHAZA (el mensaje menciona «el fin de la entrada»)	RECHAZA ✓ Error sintáctico [1:5]: se esperaba una condición (un identificador, NOT o '(') después de WHEN, pero se encontró el fin de la entrada.	Solo la palabra WHEN
L04	`WHEN a>1 THEN x y;`	ACEPTA	ACEPTA ✓	Regla mínima sin espacios alrededor del operador
L05	`WHEN ((((((((((((((((((((a > 1))))))))))))))))))) THEN x y; [20 niveles]`	ACEPTA	ACEPTA ✓	20 niveles de paréntesis anidados
L06	`WHEN s0 > 0 AND s1 > 1 AND ... AND s199 > 199 THEN x y; [200 comparaciones]`	ACEPTA	ACEPTA ✓	200 comparaciones unidas por AND
L07	`WHEN s0 > 0 THEN activar r0; ... WHEN s99 > 99 THEN activar r99; [100 reglas]`	ACEPTA	ACEPTA ✓	Programa con 100 reglas
L08	`when temperatura > 28 then encender ventilador;`	ACEPTA	ACEPTA ✓	Palabras reservadas en minúsculas (el lexer es insensible a mayúsculas)
L09	`WHEN sensor_1 > 0.5 THEN abrir valvula_2 100;`	ACEPTA	ACEPTA ✓	Identificadores con _ y dígitos; decimal
L10	`WHEN when > 1 THEN x y;`	RECHAZA (el mensaje menciona «la palabra reservada WHEN»)	RECHAZA ✓ Error sintáctico [1:6]: se esperaba una condición (un identificador, NOT o '(') después de WHEN, pero se encontró la palabra reservada WHEN.	Palabra reservada usada como identificador
L11	`WHEN a > 2abc THEN x y;`	RECHAZA (el mensaje menciona «el identificador 'abc'»)	RECHAZA ✓ Error sintáctico [1:11]: se esperaba la palabra reservada THEN (o un conector AND / OR) después de la condición, pero se encontró el identificador 'abc'.	Número pegado a letras: se separa en NUM + ID
L12	`WHEN a > 1\nTHEN x\n;`	RECHAZA (el mensaje menciona «línea 3»)	RECHAZA ✓ Error sintáctico [3:1]: se esperaba el dispositivo u objeto (identificador) después de la acción 'x', pero se encontró el símbolo ';'.	Error en la línea 3 (acción sin dispositivo, con saltos de línea)
L13	`WHEN a > 1. THEN x y;`	RECHAZA (el mensaje menciona «símbolo no reconocido '.'»)	RECHAZA ✓ Error léxico [1:11]: símbolo no reconocido '.'.	Decimal incompleto '1.'
L14	`WHEN ((a > 1)) THEN x y;`	ACEPTA	ACEPTA ✓	Paréntesis redundantes

Análisis de los casos límite. Cubren: entrada vacía y solo espacios (L01–L02); regla truncada (L03); regla mínima sin espacios (L04); anidamiento profundo de 20 paréntesis (L05); 199 conectores `AND` entre 200 comparaciones (L06) y 100 reglas (L07), que prueban que los ciclos no desbordan la pila; mayúsculas/minúsculas (L08); identificadores con _ y dígitos y decimales (L09); palabra reservada como identificador (L10); número

pegado a letras (L11); error ubicado en la línea 3 de una entrada multilínea (L12); decimal incompleto (L13) y paréntesis redundantes (L14).

11 Conclusiones

- Se diseñó un DSL completo para automatización de una casa inteligente y se formalizó con una GLC de 24 producciones, obtenida transformando una primera versión natural (G_0) que era recursiva por la izquierda, tenía prefijos comunes y era ambigua.
- Las transformaciones (eliminación de recursión izquierda, factorización y estratificación por precedencia) llevaron a una gramática **LL(1) sin conflictos**, comprobada con FIRST, FOLLOW, conjuntos de selección y la tabla.
- Se implementaron **dos analizadores predictivos escritos a mano** (descenso recursivo y dirigido por tabla) que coinciden en los 46 casos; el primero construye un AST legible y el segundo el árbol de derivación y una traza.
- El manejo de errores entrega mensajes útiles y localizados (línea, columna, posición, puntero), sensibles al contexto.
- La extensión (`NOT`, paréntesis, acciones múltiples con parámetros) no es decorativa: modifica la gramática, FIRST/FOLLOW, la tabla, ambos parsers y las pruebas.

Limitaciones y trabajo futuro. Un solo error por ejecución (sin recuperación en modo pánico); no se analiza la semántica (p. ej. que `temperatura` sea una variable existente o que `encender` sea aplicable a `ventilador`); y las palabras reservadas son insensibles a mayúsculas, decisión heredada de la función base. Extensiones posibles: bloques `IF-ELSE`, comentarios, unidades de medida y un verificador semántico.

12 Anexos

Anexo A. Repositorio

El código fuente completo, el `README.md` (instrucciones de ejecución, dependencias, formato de entrada y ejemplos), los casos de prueba, los ejemplos y las imágenes están en el repositorio:

`https://github.com/BayronFlores/domodsl-parser-ll1`

Para reproducir: `python main.py "WHEN temperatura > 28 THEN encender ventilador;"`, `python ejecutar_pruebas.py` y `python -m unittest -v`.

El código fuente completo, el `README.md` y los archivos auxiliares (`ejemplos/`, `docs/`) están en el repositorio del Anexo A. Como material complementario, las capturas también están en una carpeta de Google Drive:

`https://drive.google.com/drive/folders/1D8J06A_bD95Yj7PwBkc-qyCM0PciW3j4?usp=sharing`

En la sección 7 se incluyen los extractos principales del código del parser.